Technical Library - Editor Guidelines

Copernicus Land Monitoring Serviceross references

Author: **European Environment Agency (EEA)**

Date: **2026-05-26**

Version: **1.2.0**

Table of contents

1. How This Works 1
    1.1 Your project and the central library 1
    1.2 The publish flow 1
    1.3 The document lifecycle 1
2. Starting a New Document 2
    2.1 From a template 2
    2.2 From scratch 2
    2.3 Migrating an existing document 2
3. The YAML Header 3
    3.1 Required fields 3
    3.2 Optional fields 3
    3.3 System-managed fields 3
4. Writing Content 3
    4.1 Image folder convention 3
    4.2 Page breaks 3
    4.3 Complex table layouts 4
5. Editing and Previewing Locally 4
    5.1 Open the project in RStudio 4
    5.2 Preview your document 4
    5.3 Render the full project 5
6. Publishing to Develop 5
7. Folder and File Structure 6
8. File Naming, Versioning, and Changelog 6
    8.1 The version suffix 6
    8.2 When to create a new major version 6
    8.3 Don’t rename after publishing 7
    8.4 The change log 7
9. Restoring Previous Versions 7
    9.1 Commands 7
    9.2 When to use it 7
10. Release Path 8
11. Change Log 8

Contact:

European Environment Agency (EEA)
Kongens Nytorv 6
1050 Copenhagen K

Denmark
https://land.copernicus.eu/

This guide is for editors writing and publishing documents in the CLMS Technical Library. It covers the day-to-day workflow: starting a document from a template, previewing it locally, publishing it to the library, and keeping it in step with the central repository.

> **Note**
>
> This guide assumes your project is already integrated with the Technical Library - the `DOCS/` subtree is in place and the Git aliases are configured. If that setup isn't done yet, see the **Setup Guideline** first.

1. How This Works

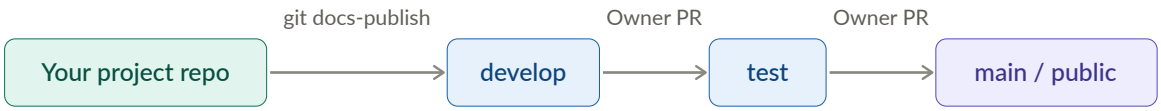
1.1 Your project and the central library

Your project repository contains a `DOCS/` folder that is both part of your project and part of the central CLMS Technical Library. The connection works as a **git subtree**: your documents live in your repo, but `git docs-publish` merges them into the library's `develop` branch.

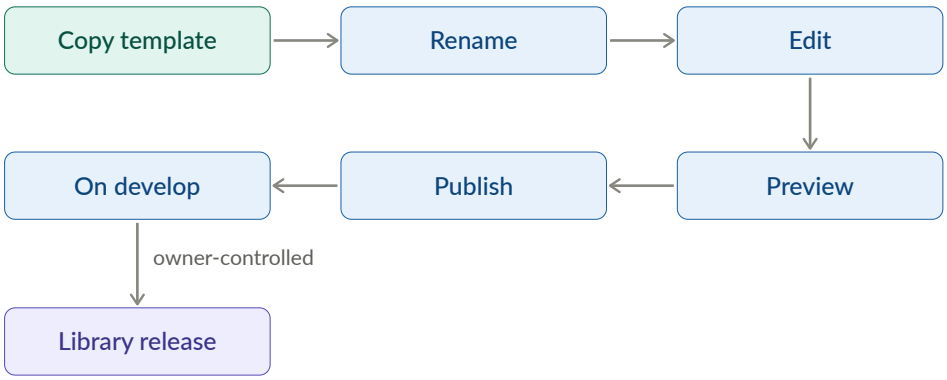
That means you own the source. You edit locally, commit to your project, and publish when ready. The library pulls in documents from all projects.

1.2 The publish flow

Publishing to `develop` is not a public release. The public-facing library only changes when a Technical Library owner promotes changes from `develop` to `test` to `main`. As an editor, your work stops at `develop`.



1.3 The document lifecycle



Major version bumps are your decision - you create a new file with a bumped `_v<N>` suffix. Minor and patch versions are AI-assigned at release time based on the scope of changes.

2. Starting a New Document

2.1 From a template

Pick the template that matches your document type:

- `CLMS_Template_ATBD.qmd` - Algorithm Theoretical Basis Document, for the scientific and technical foundations of a data product
- `CLMS_Template_PUM.qmd` - Product User Manual, for how users access and interpret a product

Once you've chosen:

1. Copy the template into `DOCS/` (not sure what that looks like? See **Folder and File Structure**)
2. Rename it using this pattern: `CLMS_<Product-Name>_<Document-Type>_v<Major-Version>.qmd`

   For example: `CLMS_HRL-Forest_PUM_v1.qmd`

   Use hyphens within the product name, not underscores. Start every new document at `_v1`. The document type is either `ATBD` or `PUM`.
3. Create a matching `-media/` folder for images: `CLMS_HRL-Forest_PUM_v1-media/`
4. Edit in place, following the structure the template provides

The templates include a pre-filled YAML header, required sections, and commented guidance throughout. Try to keep the structure intact - it makes a real difference to how consistently documents behave across the library.

2.2 From scratch

If neither template fits, create a new `.qmd` file directly in `DOCS/`, following the same naming pattern, create a matching `-media/` folder, and add the required YAML header manually (see **The YAML Header**).

> **Tip**
>
> Templates save time even when they don't feel like an exact fit - the YAML header and section structure are the main things you'd otherwise need to set up manually.

2.3 Migrating an existing document

Migrate when you already have substantial content in an existing `.qmd` file; start from a template when you don't. An existing `.qmd` can be dropped into the library as-is, but it needs to be adapted first.

1. **Copy.** Place the existing `.qmd` in `DOCS/<product>/` and rename it with the pattern above, e.g. `CLMS_HRL-Forest_PUM_v1.qmd`.
2. **Trim the YAML header** down to `title`, `subtitle`, `category`, and `date` if needed.
3. **Move the images** into the sibling `<document-name>—media/` folder and make every reference relative. Check that the file extension case matches what is on disk: a `.PNG` referenced as `.png` renders locally but breaks the published build.
4. **Verify** with a local render and the frontmatter check before publishing:

```
quarto render DOCS/<product>/<document>_v1.qmd
python3 .github/scripts/validate_qmd_files.py
```

3. The YAML Header

If you copied a template, the header is pre-filled - just update `title`, `subtitle`, and `date`.

3.1 Required fields

```yaml
---
title: HRL Forest
subtitle: Copernis Land Monitoring Service
category: products
date: "2026-01-27"
---
```

- `title` - short product name, appears as the document title
- `subtitle` - Copernis Land Monitoring Service
- `category` - one of `products`, `guidelines`, `reports`, `uncategorized`, `non-browsable`
- `date` - publication or last-updated date in ISO format e.g. '2026-05-26'

3.2 Optional fields

- `type` - how the document is built. Leave it out for ordinary documents. Set `type: dashboard` for an interactive page: it renders to HTML only - no PDF, no table of contents, no version or changelog. New types are added by a Technical Library owner.
- `product-name` - used internally for metadata and indexing
- `template-version` - records which template version was used as the starting point. Set automatically when copying a template; don't edit it manually.

3.3 System-managed fields

Leave these out of your YAML header entirely - the publishing system handles them:

- `keywords` - AI-generated at publish time based on document content. Anything you add gets overwritten.
- `version` - carried by the filename suffix, not the YAML. No version field belongs in the header.

4. Writing Content

Markdown basics - headings, lists, links, images, tables, equations, footnotes, diagrams - are covered in the Quarto Markdown guide. This chapter covers only what's specific to this system.

4.1 Image folder convention

Images go in `<your-document-name>—media/`, referenced with relative paths:

```
![Processing workflow](CLMS_HRL-Forest_PUM_v2-media/workflow.png)
```

4.2 Page breaks

Use the Quarto shortcode to insert a page break in PDF output:

```
{{< pagebreak >}}
```

Note that `---` renders as a horizontal rule, not a page break.

4.3 Complex table layouts

For tables with merged cells, nested content, or custom styling, use HTML tables directly in your `.qmd` file. Both the HTML output and Typst-rendered PDF handle them well.

Standard Markdown tables have limited formatting options. HTML tables let you control exactly how things look - column widths, cell colours, text alignment, borders - using inline CSS:

```
```{=html}
<table style="width: 100%; border-collapse: collapse;">
  <thead>
    <tr style="background-color: #2c3e50; color: white;">
      <th colspan="2" style="padding: 10px; text-align: center;">Product Overview</th>
    </tr>
  </thead>
  <tbody>
    <tr>
      <td style="padding: 8px; border: 1px solid #ccc; font-weight: bold;">HRL Forest</td>
      <td style="padding: 8px; border: 1px solid #ccc;">Active</td>
    </tr>
    <tr style="background-color: #f2f2f2;">
      <td style="padding: 8px; border: 1px solid #ccc; font-weight: bold;">Water Bodies</td>
      <td style="padding: 8px; border: 1px solid #ccc;">Active</td>
    </tr>
  </tbody>
</table>
```
```

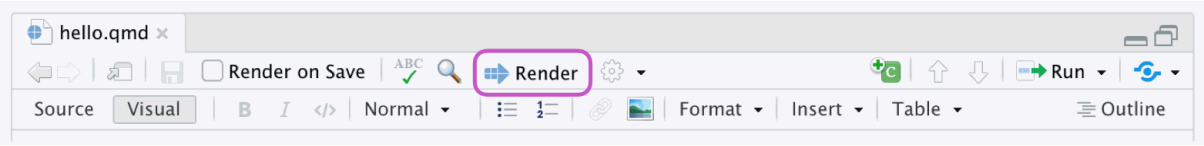
5. Editing and Previewing Locally

5.1 Open the project in RStudio

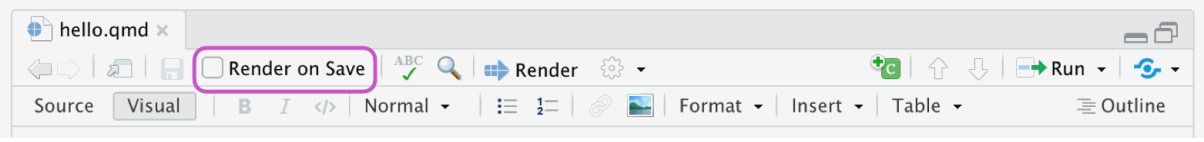
1. Open RStudio
2. **File > Open Project...** and select the root folder of your project
3. Navigate to `DOCS/` and open the `.qmd` file you want to edit

5.2 Preview your document

Click the **Render** button at the top of the editor pane to render HTML or PDF:



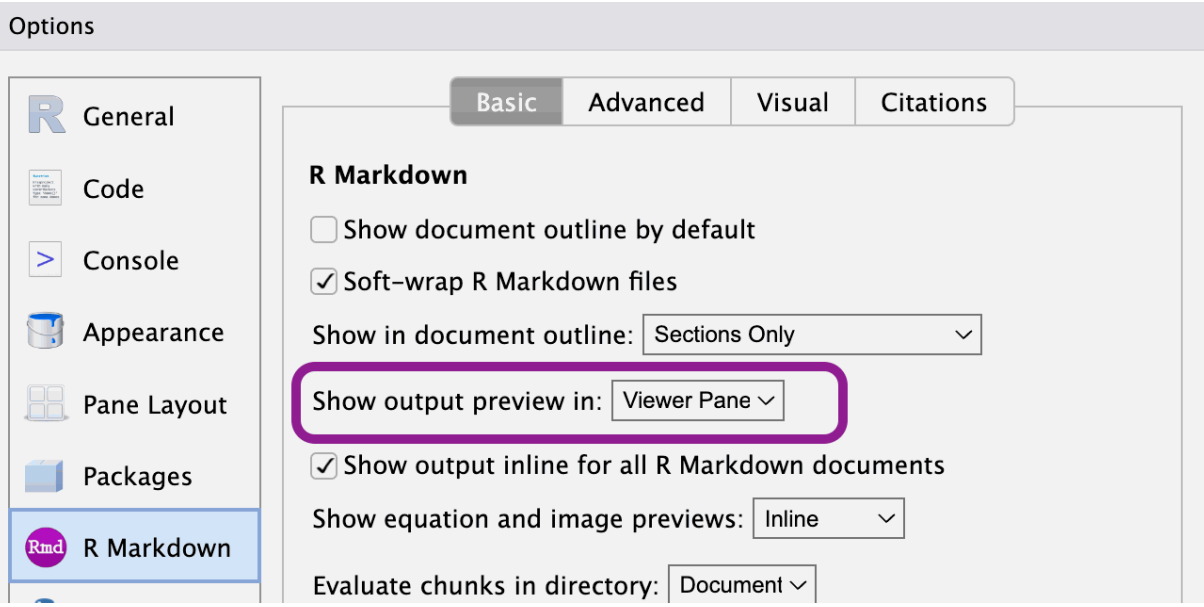
Or enable **Preview on Save** in the toolbar to re-render on every save:



The rendered output appears in the **Viewer** tab in the lower-right pane:



If the Viewer tab isn't showing rendered output, go to **Tools > Global Options > R Markdown** and set **Show output preview in** to **Viewer Pane**:



5.3 Render the full project

To render all documents and preview the complete library locally:

```
git docs-preview
```

> **Tip**
>
> The repository ships with the project config renamed to `_quarto-not-used.yaml` so RStudio operates in single-file mode by default - faster for day-to-day editing. When you need to render the full project, rename it to `_quarto.yaml`, run `quarto render`, then rename it back.

6. Publishing to Develop

```
git add DOCS/
git commit -m "docs: update [brief description]"
git docs-publish
```

After the pipeline runs, check your document at: https://eea.github.io/CLMS_documents/develop/index.html

Navigation, cross-links, and the sidebar TOC can behave differently than in your local RStudio preview - worth a quick look before calling it done.

7. Folder and File Structure

```
DOCS/
├── _meta/                          # Scripts, config, metadata - do not edit
├── includes/                       # Quarto include files - do not edit
├── templates/                      # Document templates - do not edit
├── theme/                          # Styling definitions - do not edit
├── _quarto.yml                     # Project-wide Quarto config
├── CLMS_HRL-Forest_PUM_v2.qmd
├── CLMS_HRL-Forest_PUM_v2-media/
├── CLMS_Water-Bodies_ATBD_v1.qmd
├── CLMS_Water-Bodies_ATBD_v1-media/
└── ...
```

Your work lives in `DOCS/` directly: one `.qmd` file per document, plus a matching `-media/` folder for images and diagrams. The four managed subdirectories (`_meta/`, `includes/`, `templates/`, `theme/`) are updated centrally - don't edit them.

8. File Naming, Versioning, and Changelog

8.1 The version suffix

The `_v<N>` suffix in the filename is the major version - the only version number you control. Minor and patch versions are assigned automatically by AI at library release time based on the scope of changes: small corrections get a patch bump (`2.3.5 → 2.3.6`), new or revised sections get a minor bump (`2.3.0 → 2.4.0`). A new file `CLMS_HRL-Forest_PUM_v3.qmd` starts at `3.0.0` at its first release. You never set a version number in YAML.

Version numbers are assigned only when your changes reach `main`, i.e. at the public release. Publishing to `develop`, or promoting to `test`, assigns nothing - the document keeps its previous version until the release build runs.

8.2 When to create a new major version

Most changes are an edit to the existing file - the system bumps the minor or patch version for you. Create a new `_v<N>` file only when the **old version has to stay available exactly as it is**, for example when you:

- work on a new product release
- need the earlier release to keep being citable as a stable reference

Creating one is a fork, not an update. Copy the file and its media folder, and from that point the two documents are independent: separate version numbers, separate change logs, and no link between them.

```
cp DOCS/CLMS_HRL-Forest_PUM_v2.qmd DOCS/CLMS_HRL-Forest_PUM_v3.qmd
cp -r DOCS/CLMS_HRL-Forest_PUM_v2-media DOCS/CLMS_HRL-Forest_PUM_v3-media
```

The copied filename must carry the new major version (`_v3` above); the new document then starts at `N.0.0` at its first release, while the old one keeps its own history.

8.3 Don't rename after publishing

The filename becomes part of the document's URL. Renaming breaks links and cross-references. If a significant revision is needed, create a new major version rather than renaming.

8.4 The change log

You don't write the change log. The system generates it at release time and appends a **Change Log** section to the end of your document - a table of date, version, and summary entries:

Date	Version	Summary
2026-11-06	2.1.0	Reworded the coverage section and corrected the tile table
2026-07-24	2.0.0	First published version

At each release - that is, when a change reaches `main`, in the same step that assigns the version number - the pipeline compares the document against its last release and writes a short summary of what changed, storing the entry under the new version. A document with no previous release gets an initial entry instead.

The stored history is what the library build renders, newest entry first, as the **Change Log** section appended after your content. Because it comes from stored history rather than the current diff, it shows up in every built version of the page (`develop`, `test` and `main` alike) - you just won't see your own change there until it has been released. At most the last 20 entries are kept.

What this means in practice:

- **Nothing to add to the source.** There is no `changelog` field and no section to maintain.
- **Write descriptive commits.** The summary is derived from the diff, so focused changes produce a clearer entry.
- **It's absent from your local preview.** `quarto render` doesn't run the library build, so the section only appears on a built page. This document has one too - see its Change Log at the end of the published page.
- **Some documents have none.** Interactive pages (`type: dashboard`) opt out of the change log. Leave `type` unset unless the document really is one.

9. Restoring Previous Versions

Restore pulls a published version of a document into your local working copy. It doesn't roll back the library - think of it as "pick up from version X," not "undo to version X."

9.1 Commands

```
git docs-restore --list-all                                  # all published documents
git docs-restore --list DOCS/CLMS_HRL-Forest_PUM_v2.qmd      # versions of one file
git docs-restore DOCS/CLMS_HRL-Forest_PUM_v2.qmd 2.0.0       # specific version
git docs-restore --latest DOCS/CLMS_HRL-Forest_PUM_v2.qmd    # latest published version
```

9.2 When to use it

Recover deleted content. You removed a section locally and need it back. Restore the last published version, copy out what you need, then carry on with your current version.

Something went wrong in a recent edit. Restore an earlier version (e.g. `2.1.0`) and work forward from there rather than trying to untangle the current state.

Resurrect a deleted file. If a file was removed from the project but still exists in the library history, `--list-all` will find it. Restore brings it back into `DOCS/` as if nothing happened.

10. Release Path

Once you publish to `develop`, the rest of the process belongs to Technical Library owners. Getting from develop to public requires two separate pull requests, each reviewed and approved by a second owner before it merges.

Stage	URL
develop	https://eea.github.io/CLMS_documents/develop/index.html
test	https://eea.github.io/CLMS_documents/test/index.html
main (public)	https://library.land.copernicus.eu

At each release, the system assigns version numbers automatically based on what changed.

11. Change Log

Date	Version	Summary
2026-06-03	1.2.0	• Major restructuring and rewrite of the manual to focus on practical editing aspects for the CLMS Technical Library. • Updated document title to 'Editor Guide' and revised its description. • Removed extensive sections on Quarto and Markdown basics, software installation, detailed rendering, and troubleshooting. • Added new simplified sections on document lifecycle, publishing flow, folder and file structure, naming conventions, versioning, and restoring previous versions. • Updated the document date.
2026-02-10	1.1.0	• Extended explanation for `git docs-restore` functionality. • Added major version upgrade process explanation. • Added more precise instructions. • Minor edits to wording in various sections.
2025-12-03	1.0.0	Initial release